SOFTWARE ENGINEERING LAB
LAB 5 – STATIC CODE ANALYSIS
Recruitment Tracker Application

Name	Kirthana S
SRN	PES1UG24AM136
Section	5 C

1. Introduction

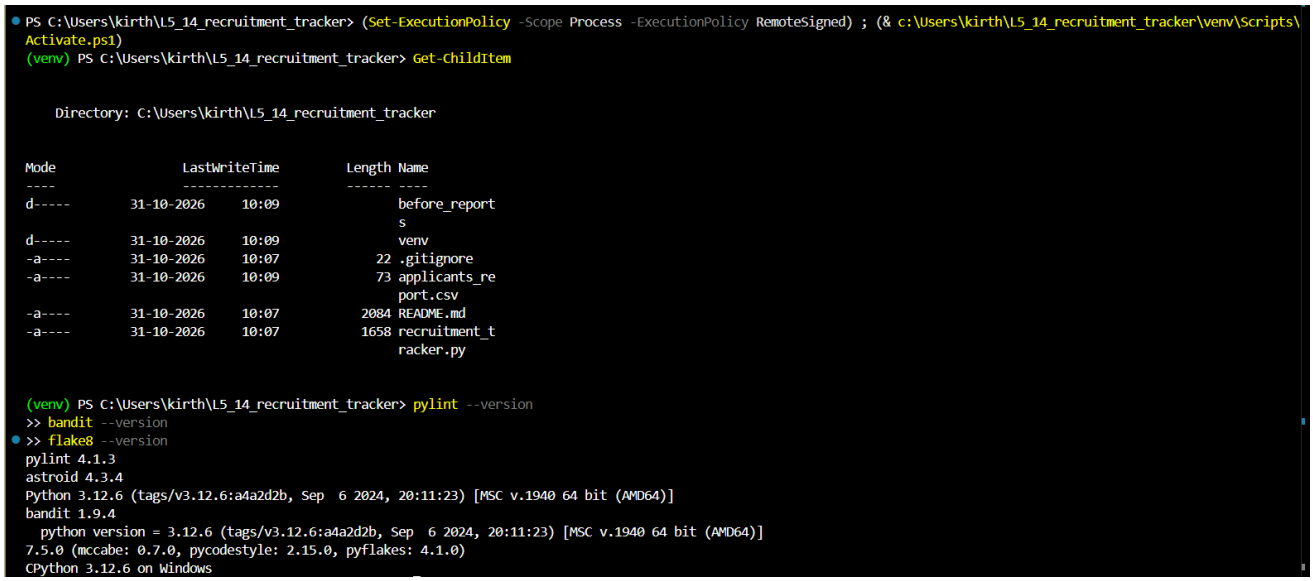
This report presents the work completed for Lab 5 on the Recruitment Tracker program. The original Python file was checked using Pylint, Bandit and Flake8. The issues reported by these tools were reviewed, corrections were made, and the tools were run again to check the updated code.

2. Objectives

- Analyse the assigned Python program using Pylint, Bandit and Flake8.
- Identify code-quality, security and formatting issues reported by the tools.
- Correct the reported issues while preserving recruitment-tracker functionality.
- Re-run all three tools and record the final results.
- Document the findings, fixes, execution evidence and reflections.

3. Tools and Environment

Tool / Component	Purpose / Version
Python	Python 3.12.6 (reported by Bandit)
Pylint	4.1.3 – code quality and static analysis
Bandit	1.9.4 – Python security issue detection
Flake8	7.5.0 – style and formatting checks
Editor / OS	Visual Studio Code / Windows PowerShell



Python virtual environment and static analysis tools.

4. Application Overview

The Recruitment Tracker is a Python program that stores applicant details in an SQLite database. Its demonstrated features include adding applicants, shortlisting applicants based on a salary condition, updating applicant status and counting applicants by status. The file analysed in this lab is recruitment_tracker.py.

5. Initial Static Analysis

Before changing the program, I ran it and saved the output from all three analysis tools. These reports form the baseline for comparing the code before and after the fixes.

Tool	Initial result	Interpretation
Pylint	4.47/10	Code-quality, documentation, naming, return, exception and resource-management findings.
Bandit	3 findings	B608 SQL construction (Medium); B307 eval() (Medium); B110 try/except/pass (Low).
Flake8	7 violations	Long lines, unused variable, trailing whitespace and blank-line formatting.

5.1 Baseline Evidence

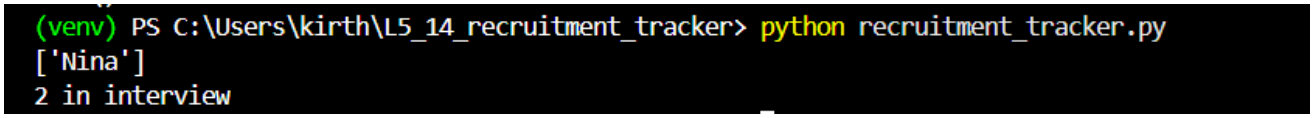


Figure 1. Original execution of the Recruitment Tracker application.

Figure 2. Pylint before corrections

```
(venv) PS C:\Users\kirth\L5_14_recruitment_tracker> pylint recruitment_tracker.py
************* Module recruitment_tracker
recruitment_tracker.py:4:0: C0301: Line too long (124/100) (line-too-long)
recruitment_tracker.py:8:0: C0301: Line too long (140/100) (line-too-long)
recruitment_tracker.py:13:0: C0301: Line too long (106/100) (line-too-long)
recruitment_tracker.py:15:26: C0303: Trailing whitespace (trailing-whitespace)
recruitment_tracker.py:1:0: C0114: Missing module docstring (missing-module-docstring)
recruitment_tracker.py:7:0: C0116: Missing function or method docstring (missing-function-docstring)
recruitment_tracker.py:7:0: C0103: Function name "addApplicant" doesn't conform to snake_case naming style (invalid-name)
recruitment_tracker.py:12:0: C0116: Missing function or method docstring (missing-function-docstring)
recruitment_tracker.py:14:12: W0123: Use of eval (eval-used)
recruitment_tracker.py:12:0: R1710: Either all return statements in a function should return an expression, or none of them should. (inconsistent-return-statements)
recruitment_tracker.py:15:4: W0612: Unused variable 'total_seen' (unused-variable)
recruitment_tracker.py:24:0: C0116: Missing function or method docstring (missing-function-docstring)
recruitment_tracker.py:24:0: C0103: Function name "updateStatus" doesn't conform to snake_case naming style (invalid-name)
recruitment_tracker.py:28:11: W0718: Catching too general exception Exception (broad-exception-caught)
recruitment_tracker.py:31:0: C0116: Missing function or method docstring (missing-function-docstring)
recruitment_tracker.py:31:0: C0103: Function name "exportReport" doesn't conform to snake_case naming style (invalid-name)
recruitment_tracker.py:32:8: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
recruitment_tracker.py:32:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
recruitment_tracker.py:38:0: C0116: Missing function or method docstring (missing-function-docstring)
recruitment_tracker.py:38:0: C0103: Function name "countByStatus" doesn't conform to snake_case naming style (invalid-name)
recruitment_tracker.py:42:0: C0116: Missing function or method docstring (missing-function-docstring)

-------------------------------------------------------------------
Your code has been rated at 4.47/10 (previous run: 4.47/10, +0.00)
```

Figure 2. Pylint analysis of the original Recruitment Tracker source code.

Figure 3. Bandit before corrections

```
(venv) PS C:\Users\kirth\L5_14_recruitment_tracker> bandit -r recruitment_tracker.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.12.6
Run started:2026-10-31 05:03:42.707940+00:00

Test results:
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Medium
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: .\recruitment_tracker.py:8:19
7       def addApplicant(name, role, expected_salary, status="applied"):
8           conn.execute(f"INSERT INTO applicants (name, role, expected_salary, status) VALUES ('{name}', '{role}', {expected_salary}, '{status}')")
9           conn.commit()

--------------------------------------------------
>> Issue: [B307:blacklist] Use of possibly insecure function - consider using safer ast.literal_eval.
   Severity: Medium   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b307-eval
   Location: .\recruitment_tracker.py:14:12
13          rows = conn.execute("SELECT name, expected_salary FROM applicants WHERE role = ?", (role,)).fetchall()
14          limit = eval(salary_rule)
15          total_seen = len(rows)

--------------------------------------------------
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b110_try_except_pass.html
   Location: .\recruitment_tracker.py:28:4
27              conn.commit()
28          except Exception:
29              pass
30

--------------------------------------------------

Code scanned:
        Total lines of code: 38
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 1
                Medium: 2
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 1
                High: 2
Files skipped (0):
```

Figure 3. Bandit security analysis of the original Recruitment Tracker source code.

Figure 4. Flake8 before corrections

```
(venv) PS C:\Users\kirth\L5_14_recruitment_tracker> flake8 recruitment_tracker.py
recruitment_tracker.py:4:80: E501 line too long (124 > 79 characters)
recruitment_tracker.py:8:80: E501 line too long (140 > 79 characters)
recruitment_tracker.py:13:80: E501 line too long (106 > 79 characters)
recruitment_tracker.py:15:5: F841 local variable 'total_seen' is assigned to but never used
recruitment_tracker.py:15:27: W291 trailing whitespace
recruitment_tracker.py:26:80: E501 line too long (93 > 79 characters)
recruitment_tracker.py:31:1: E302 expected 2 blank lines, found 1
recruitment_tracker.py:39:80: E501 line too long (100 > 79 characters)
```

Figure 4. Flake8 formatting analysis of the original Recruitment Tracker source code.

6. Issues Identified and Corrections

The initial reports identified the issues listed below. The table summarizes the findings and the corrections applied to the source code.

Issue / finding	Tool / type	Original line(s)	Description	Fix approach
SQL built using an f-string	Bandit B608 – Medium	8	User-controlled values are interpolated into an SQL statement, creating an SQL injection risk.	Use parameterized SQL placeholders and pass values separately.
Use of eval()	Bandit B307 – Medium; Pylint W0123	14	eval() executes an expression as Python code and is unsafe for untrusted input.	Replace expression evaluation with explicit, validated input handling; do not accept executable expressions.
try/except/pass and broad exception	Bandit B110; Pylint W0718	28–29	The exception is silently ignored, hiding failures and making debugging difficult.	Catch the expected exception type and handle or report it clearly.
Unused variable total_seen	Pylint W0612; Flake8 F841	15	A local variable is assigned but never used.	Remove it or use it meaningfully if required.
Missing module/function docstrings	Pylint C0114/C0116	Multiple	The module and functions lack documentation strings.	Add concise module and function docstrings.
CamelCase function names	Pylint C0103	7, 24, 31, 38	Function names do not follow the recommended snake_case convention.	Rename functions using snake_case and update their call sites.
Inconsistent return statements	Pylint R1710	12–22	A function mixes returning a value with returning no value.	Make all execution paths return a consistent value.
File opened without context manager/encoding	Pylint W1514/R1732	32	The file is opened without an explicit encoding or guaranteed closure.	Use `with open(..., encoding='utf-8')`.
Line length and whitespace	Pylint C0301/C0303; Flake8 E501/W291	Multiple	Several lines exceed configured limits or contain trailing whitespace.	Wrap long statements and remove trailing whitespace.

Blank-line formatting	Flake8 E302	31	A top-level definition does not have the required blank lines before it.	Add two blank lines before the top-level function definition

7. Final Static Analysis Results

After applying the corrections, all three static analysis tools were executed again. The results confirm that the identified issues were addressed and the application meets the required acceptance conditions

Tool	Final result (fill after rerun)	Expected acceptance condition from handout
Pylint	10.00/10	Above 9.5/10
Bandit	0 findings	All reported issues handled; target 0 findings
Flake8	0 violations	0 formatting violations

7.1 Final Tool Evidence

Figure 5. Pylint after corrections

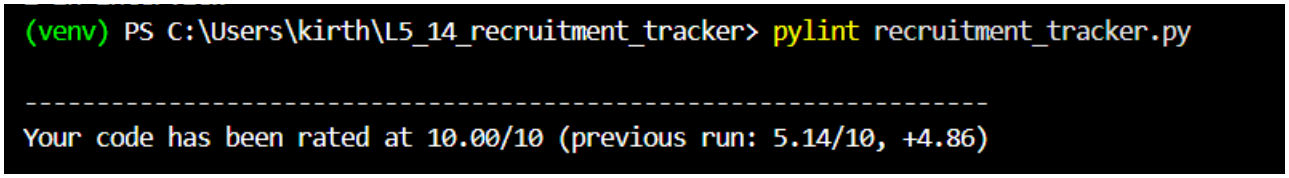


Figure 5. Pylint after corrections.

Figure 6. Bandit after corrections

```
(venv) PS C:\Users\kirth\L5_14_recruitment_tracker> bandit -r recruitment_tracker.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.12.6
Run started:2026-10-31 05:14:08.544391+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 45
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

Figure 6. Bandit analysis after security corrections.

Figure 7. Flake8 after corrections

```
(venv) PS C:\Users\kirth\L5_14_recruitment_tracker> flake8 recruitment_tracker.py
(venv) PS C:\Users\kirth\L5_14_recruitment_tracker>
```

Figure 7. Flake8 after corrections.

8. Program Execution and Functional Verification

The corrected application was executed successfully after all changes. The output confirms that applicant shortlisting and status counting continue to work as expected

Figure 8. Final program execution

```
(venv) PS C:\Users\kirth\L5_14_recruitment_tracker> python recruitment_tracker.py
['Nina']
2 in interview
```

9. Submission Structure and Version Control

The proposed folder contents are:

- README.md
- recruitment_tracker.py (cleaned source)
- issues.md
- reflection.md
- pylint_before.txt and pylint_after.txt
- bandit_before.txt and bandit_after.txt
- flake8_before.txt and flake8_after.txt

10. Reflection

What is static code analysis and why is it important?

Static code analysis examines source code without executing it. It helps identify potential defects, insecure coding practices and style problems early, improving maintainability and reliability.

What are the differences between Pylint, Bandit and Flake8?

Pylint evaluates broader code quality, including naming, documentation, unused variables and possible design issues. Bandit focuses on common security risks in Python code. Flake8 checks style and formatting, including whitespace, line length and unused code.

What issues were identified and fixed in this program?

The initial reports identified SQL query construction that may permit SQL injection, use of eval(), silent and overly broad exception handling, an unused variable, inconsistent returns, missing docstrings, non-snake_case function names, unsafe file-management practices and formatting violations. The final application addressed the identified security, code quality and formatting issues while preserving its original functionality.

How did static code analysis improve the program?

The final results demonstrate that the identified issues were resolved and that the application's original functionality was preserved.

11. Conclusion

Lab 5 involved applying Pylint, Bandit and Flake8 to the Recruitment Tracker application. The initial analysis identified code quality issues, security vulnerabilities and formatting violations. After implementing the necessary corrections, the application achieved a Pylint score of 10.00/10, zero Bandit findings and zero Flake8 violations. Successful execution confirmed that the application's functionality was preserved. The source code, analysis reports, README and reflection have been submitted to the shared team GitHub repository.